Experiment No. 9

Traffic Shaping Algorithms

Name: Parth Vishnu PRN: 24070521279

Simulation of the Leaky Bucket and Token Bucket traffic shaping algorithms using C

Objective:

- To understand the working of traffic shaping in congestion control.
- To implement the Leaky Bucket algorithm, which smoothens bursty traffic to a fixed output rate.
- To implement the Token Bucket algorithm, which allows controlled bursts using tokens.

Software/Tools Required:

- Linux (Ubuntu) system
- GCC compiler
- Text editor (vim / nano) and Terminal

Step-by-step Procedure:

Part A: Leaky Bucket Algorithm

Input: bucket capacity B, outgoing rate R, number of time units N, and incoming packets P[i] in each time unit.

Algorithm

- Initialize stored = 0 (packets currently in the bucket).
- For each time unit i from 1 to N:
  - Read the number of incoming packets P[i].
  - If stored + P[i] > B: the overflow (stored + P[i] - B) is dropped and stored = B; otherwise stored = stored + P[i].
  - If stored <= R: send all stored packets and set stored = 0; otherwise send R packets and set stored = stored - R.
  - Print the packets sent and the packets left in the bucket.
- After all time units, keep sending R packets per step until stored becomes 0.

Compile and run

```
vim exp9_279.c
gcc exp9_279.c -o exp9_279
./exp9_279
```

Part B: Token Bucket Algorithm

Input: bucket capacity B (max tokens), token generation rate R, number of time units N, and packets to send P[i] in each time unit.

Algorithm

- Initialize tokens = 0.

- For each time unit i from 1 to N:
  - Add R tokens: tokens = tokens + R. If tokens > B, set tokens = B.
  - Read the number of packets to send P[i].
  - If tokens >= P[i]: send P[i] packets and set tokens = tokens - P[i].
  - Else: send only 'tokens' packets, packets dropped = P[i] - tokens, and set tokens = 0.
  - Print the packets sent, packets dropped and tokens left.

Compile and run

```
vim exp9_2_279.c
gcc exp9_2_279.c -o exp9_2_279
./exp9_2_279
```

Observation Table

Step	Command / Action	Observation
1	Leaky Bucket: bucket size 10, outgoing rate 3, 3 time units; incoming 4, 3, 1	Packets sent 3, 3, 2; remaining in bucket 1, 1, 0
2	Token Bucket: bucket size 10, token rate 4, 3 time units; packets 3, 3, 9	Packets sent 3, 3, 6; tokens left 1, 2, 0
3	Token Bucket, time unit 3 (9 packets, only 6 tokens)	6 packets sent, 3 packets dropped

Result:

- Successfully simulated the Leaky Bucket algorithm and observed a fixed output rate.
- Successfully simulated the Token Bucket algorithm and observed bursts handled using tokens.

Conclusion:

- Leaky Bucket keeps a fixed outflow rate and drops the excess packets beyond the bucket capacity.
- Token Bucket allows bursty traffic as long as tokens are available; excess packets are dropped.

Output / Screenshots

Leaky Bucket Algorithm

```c
#include <stdio.h>

int main() {
    int bucket_size, n, i, incoming, outgoing, stored = 0;

    printf("Leaky Bucket Algorithm Simulation\n");
    printf("Enter bucket size (capacity): ");
    scanf("%d", &bucket_size);

    printf("Enter outgoing rate: ");
    scanf("%d", &outgoing);

    printf("Enter number of packets (time units): ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nTime unit %d:\n", i + 1);
        printf("Enter number of incoming packets: ");
        scanf("%d", &incoming);

        if(incoming + stored > bucket_size) {
            printf("Bucket overflow! Packet loss = %d\n", incoming + stored - bucket_size);
            stored = bucket_size;
        } else {
            stored += incoming;
        }

        if(stored <= outgoing) {
            printf("Packets sent: %d\n", stored);
            stored = 0;
        } else {
            printf("Packets sent: %d\n", outgoing);
            stored -= outgoing;
        }

        printf("Packets remaining in bucket: %d\n", stored);
    }

    // Drain remaining packets
    while(stored > 0) {
        printf("\nDraining remaining packets...\n");
        if(stored <= outgoing) {
            printf("Packets sent: %d\n", stored);
            stored = 0;
        } else {
            printf("Packets sent: %d\n", outgoing);
            stored -= outgoing;
        }
        printf("Packets left in bucket: %d\n", stored);
    }

    printf("\nSimulation complete.\n");
    return 0;
}
~
```

Fig 1: Leaky Bucket - code

```
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ vim exp9_279.c
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ gcc exp9_279.c -o exp9_279
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ ./exp9_279
Leaky Bucket Algorithm Simulation
Enter bucket size (capacity): 10
Enter outgoing rate: 3
Enter number of packets (time units): 3

Time unit 1:
Enter number of incoming packets: 4
Packets sent: 3
Packets remaining in bucket: 1

Time unit 2:
Enter number of incoming packets: 3
Packets sent: 3
Packets remaining in bucket: 1

Time unit 3:
Enter number of incoming packets: 1
Packets sent: 2
Packets remaining in bucket: 0

Simulation complete.
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$
```

Fig 2: Leaky Bucket - output

Token Bucket Algorithm

```c
#include <stdio.h>

int main() {
    int bucket_size, n, rate, tokens = 0, incoming, i;

    printf("Token Bucket Algorithm Simulation\n");
    printf("Enter bucket size (capacity): ");
    scanf("%d", &bucket_size);

    printf("Enter token generation rate: ");
    scanf("%d", &rate);

    printf("Enter number of packets (time units): ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nTime unit %d:\n", i + 1);

        tokens += rate;
        if(tokens > bucket_size) {
            tokens = bucket_size;
        }

        printf("Enter number of packets to send: ");
        scanf("%d", &incoming);

        if(tokens >= incoming) {
            printf("Packets sent: %d\n", incoming);
            tokens -= incoming;
        } else {
            printf("Only %d tokens available, packets sent: %d, packets dropped: %d\n",
                   tokens, tokens, incoming - tokens);
            tokens = 0;
        }

        printf("Tokens left: %d\n", tokens);
    }

    printf("\nSimulation complete.\n");
    return 0;
}
```

Fig 3: Token Bucket - code

```
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ vim exp9_2_279.c
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ gcc exp9_2_279.c -o exp9_2_279
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ ./exp9_2_279
Token Bucket Algorithm Simulation
Enter bucket size (capacity): 10
Enter token generation rate: 4
Enter number of packets (time units): 3

Time unit 1:
Enter number of packets to send: 3
Packets sent: 3
Tokens left: 1

Time unit 2:
Enter number of packets to send: 3
Packets sent: 3
Tokens left: 2

Time unit 3:
Enter number of packets to send: 9
Only 6 tokens available, packets sent: 6, packets dropped: 3
Tokens left: 0

Simulation complete.
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$
```

Fig 4: Token Bucket - output